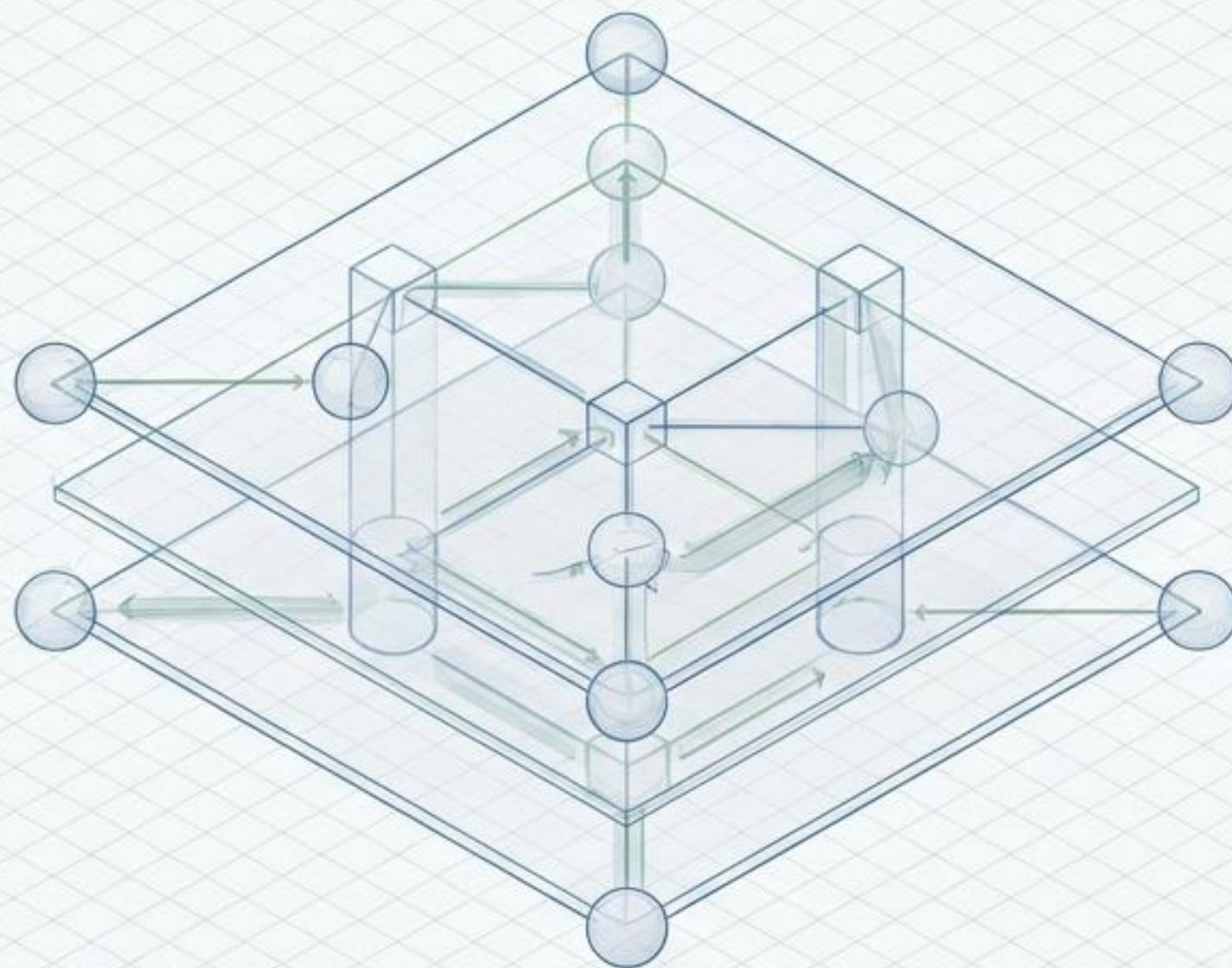




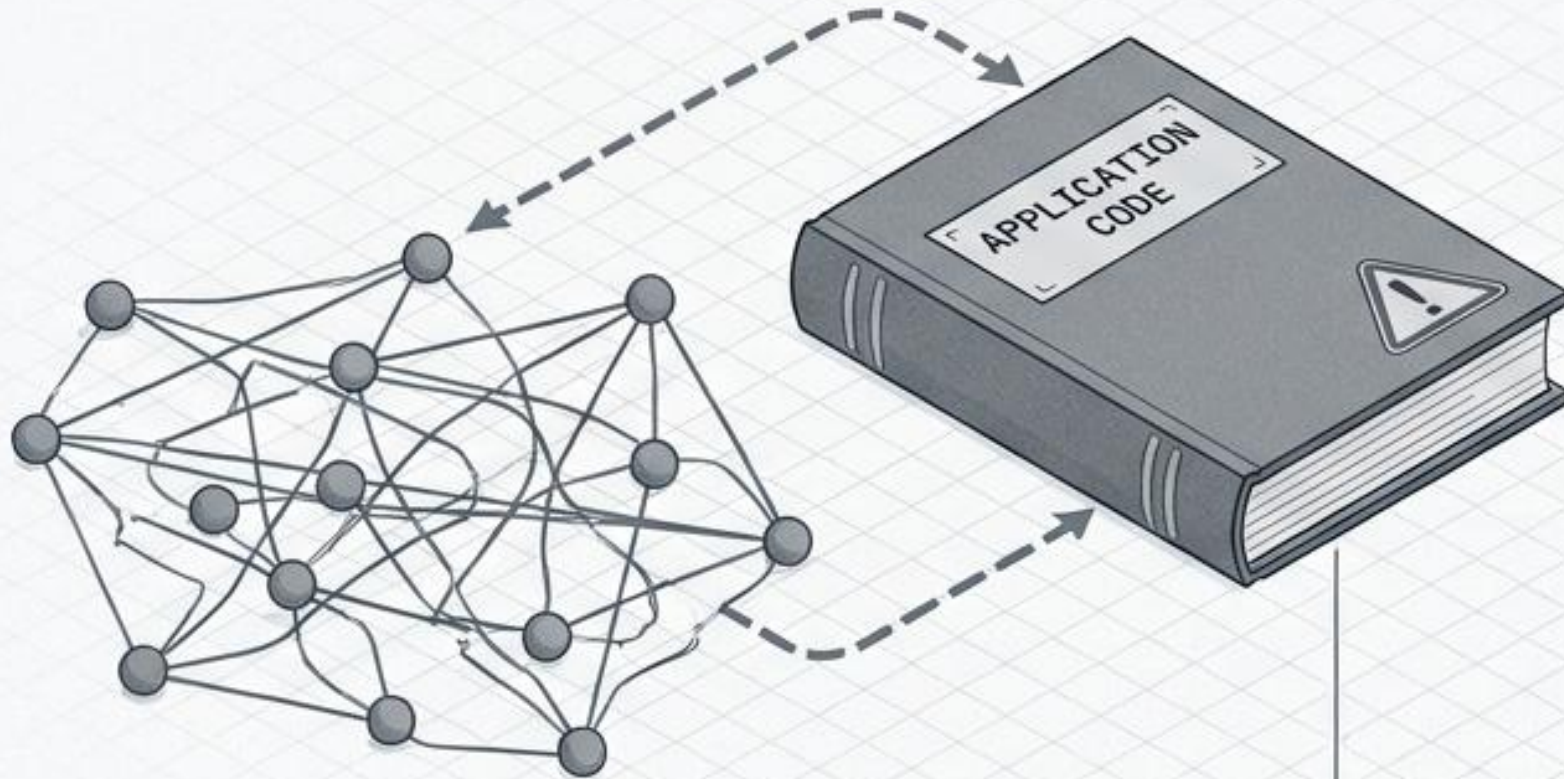
Organizing Principles for Building Knowledge Graphs

Evolving from raw connections to smart, self-describing data architectures.

Author: dr Zbigniew Galar

The Problem with Plain Graphs: Hidden Logic

Plain Old Graph

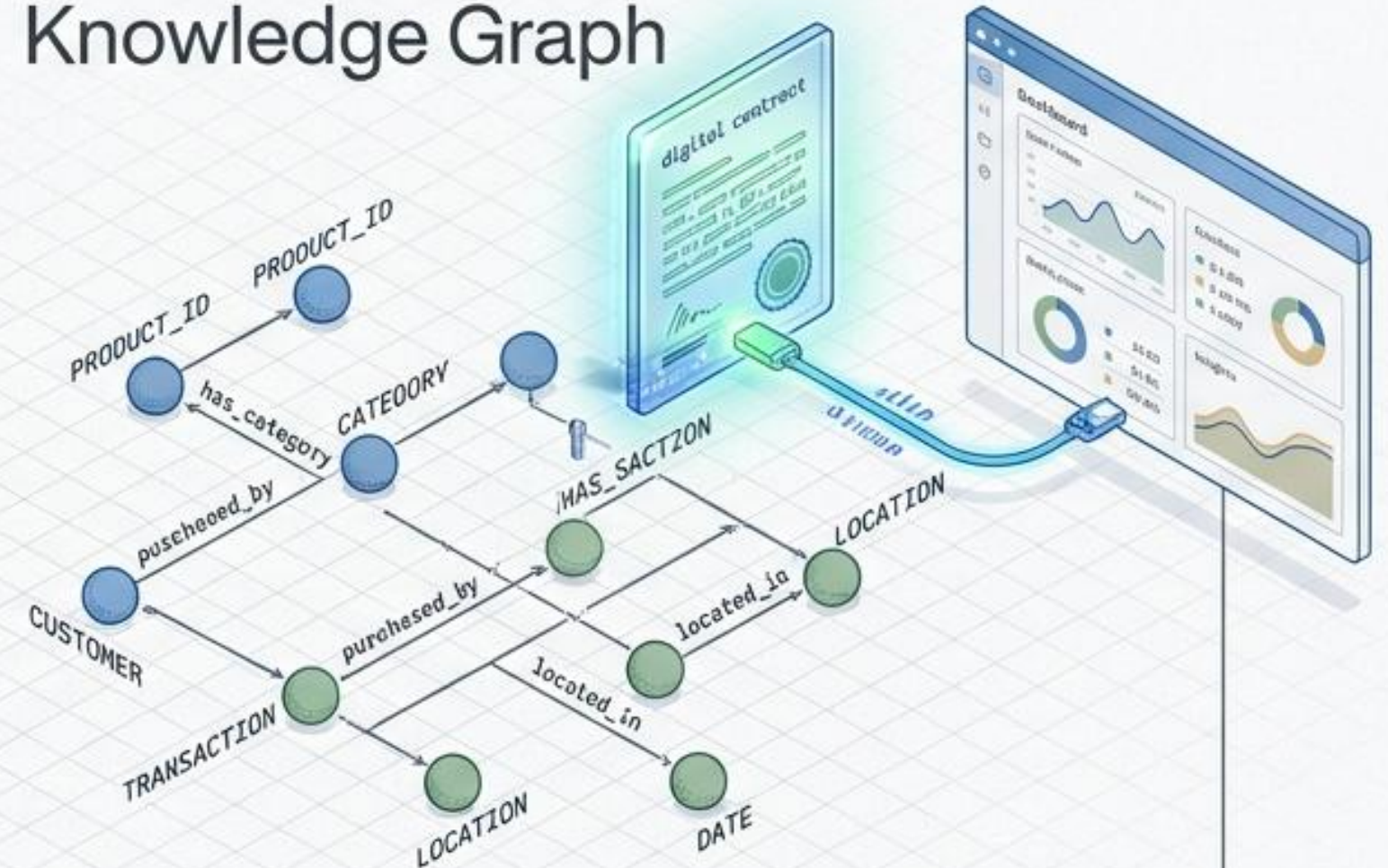


1

Plain Old Graph.

Meaning is locked away in external application code. To interpret the data, you must "read the manual" (the queries and programs). If the engineer leaves, users must reverse-engineer the logic.

Knowledge Graph

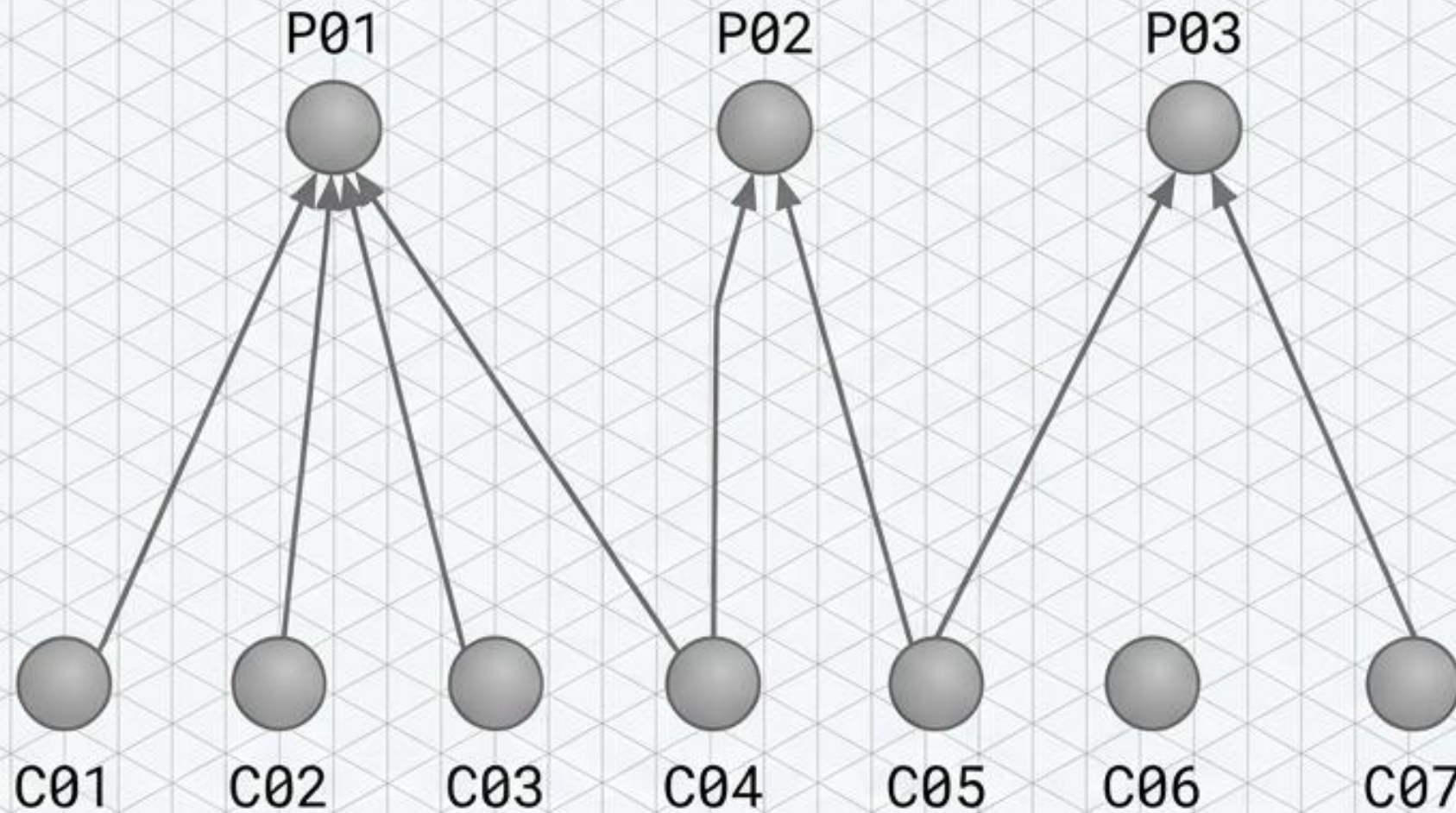


1

Knowledge Graph.

Data becomes self-describing. By embedding an organizing principle into the data itself, the graph forms a computational contract. Visualization tools can interpret the network immediately, without human intervention.

Level 0: The Plain Old Graph



Core Concept

Connections exist, but interpretation requires external knowledge. The organizing principle is absent.

Capabilities

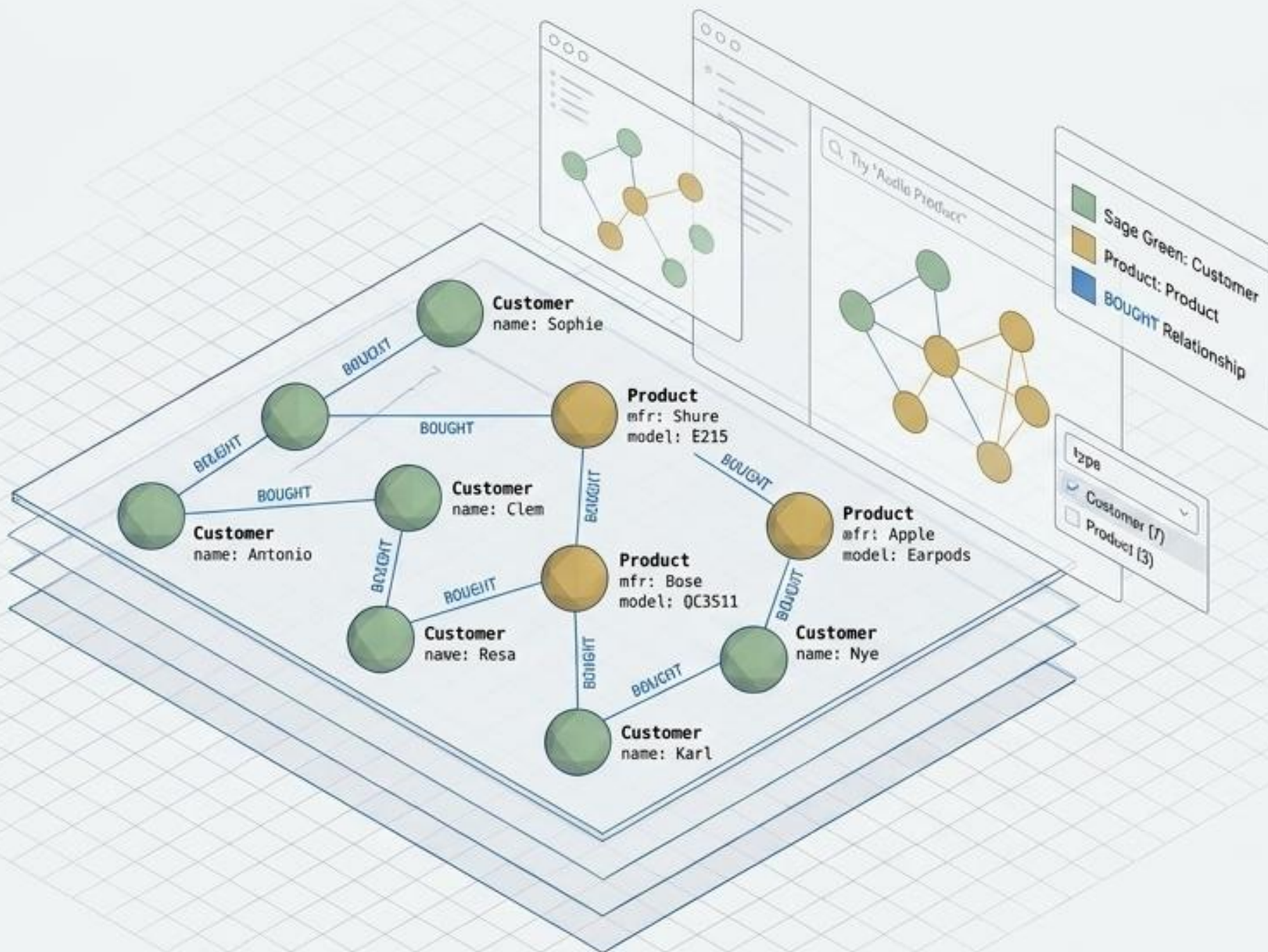
Can answer directional queries ("What did they buy?") and compute basic metrics by counting incoming relationships.

The Limitation

`PRODUCT_POPULARITY = indegree(P01)`

A data scientist running a basket analysis cannot interpret the structural meaning without domain guidance hidden in the application.

Level 1: Labeled Property Graphs (The Contract)



The Upgrade

Adding node labels, relationship types, and key-value properties. Labels act as tags describing a node's role, providing a clear structural contract for software.

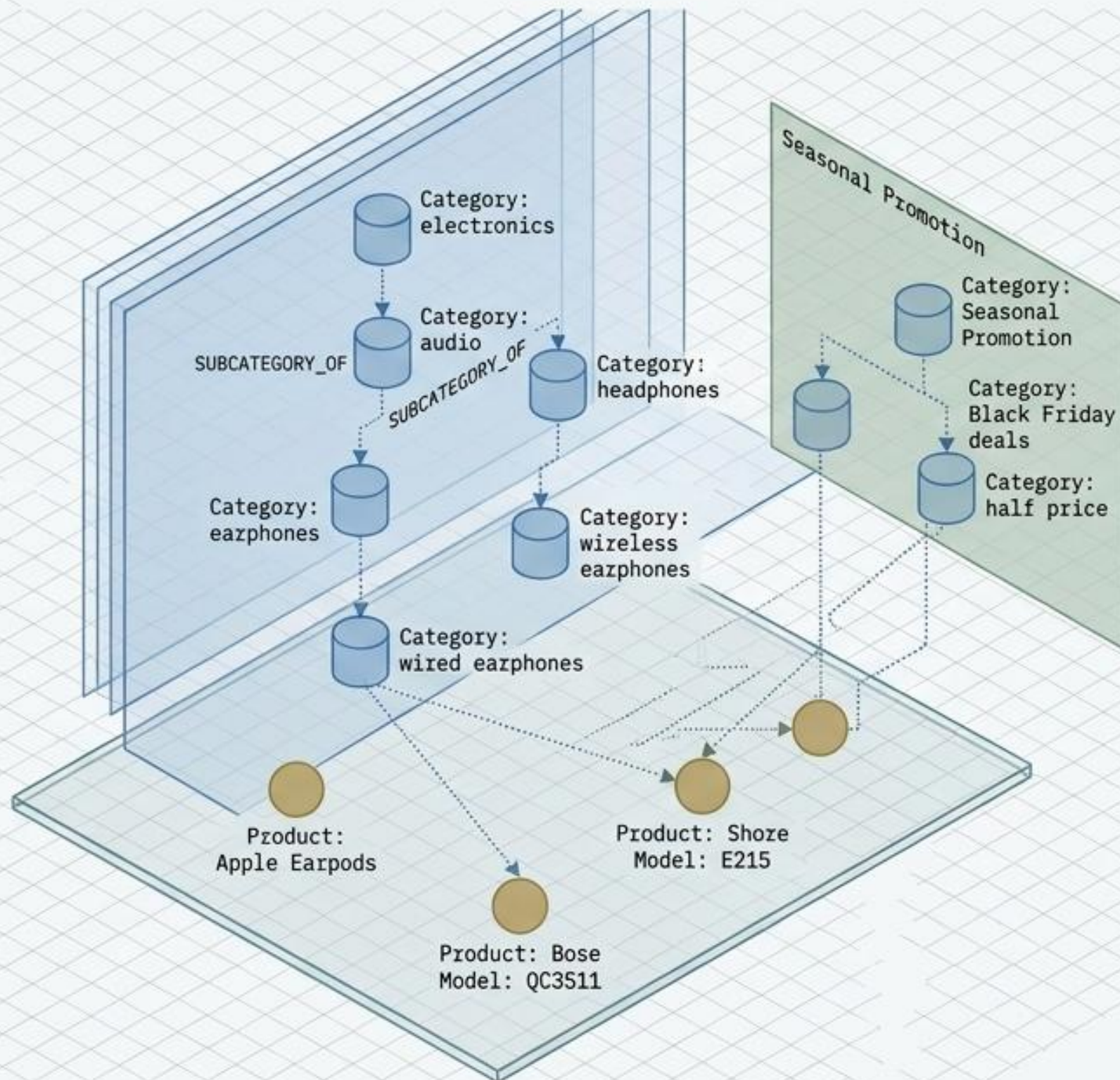
Business Capability

Immediate visual exploration and data extraction. Tools can automatically group all 'Customers' or filter for 'Audio Products' without hard-coded domain logic.

The Limitation

Labels lack associativity. The graph cannot infer that a wired headphone is a specialization of an audio product.

Level 2: Taxonomies for Hierarchy



The Upgrade

Organizing categories via broader/narrower (IS_A) hierarchies. Multiple hierarchies—like standard catalog vs. seasonal deals—can coexist and layer dynamically as graph data.

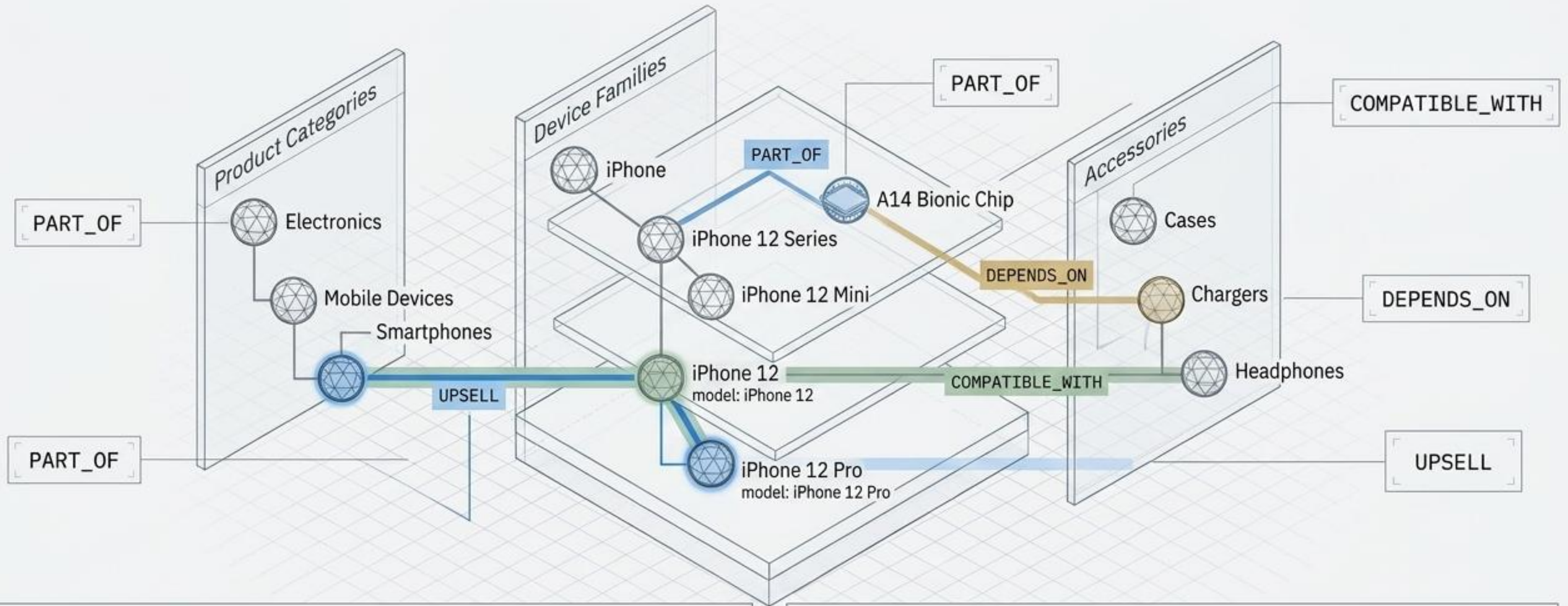
Business Capability

Semantic similarity and dynamic substitution. Enables algorithms to accurately group similar items (e.g., standard taxonomy metrics like Leacock-Chodorow) regardless of domain.

The Output

Software can execute “x is a kind of y” reasoning without manual programming.

Level 3: Ontologies for Multilevel Relationships



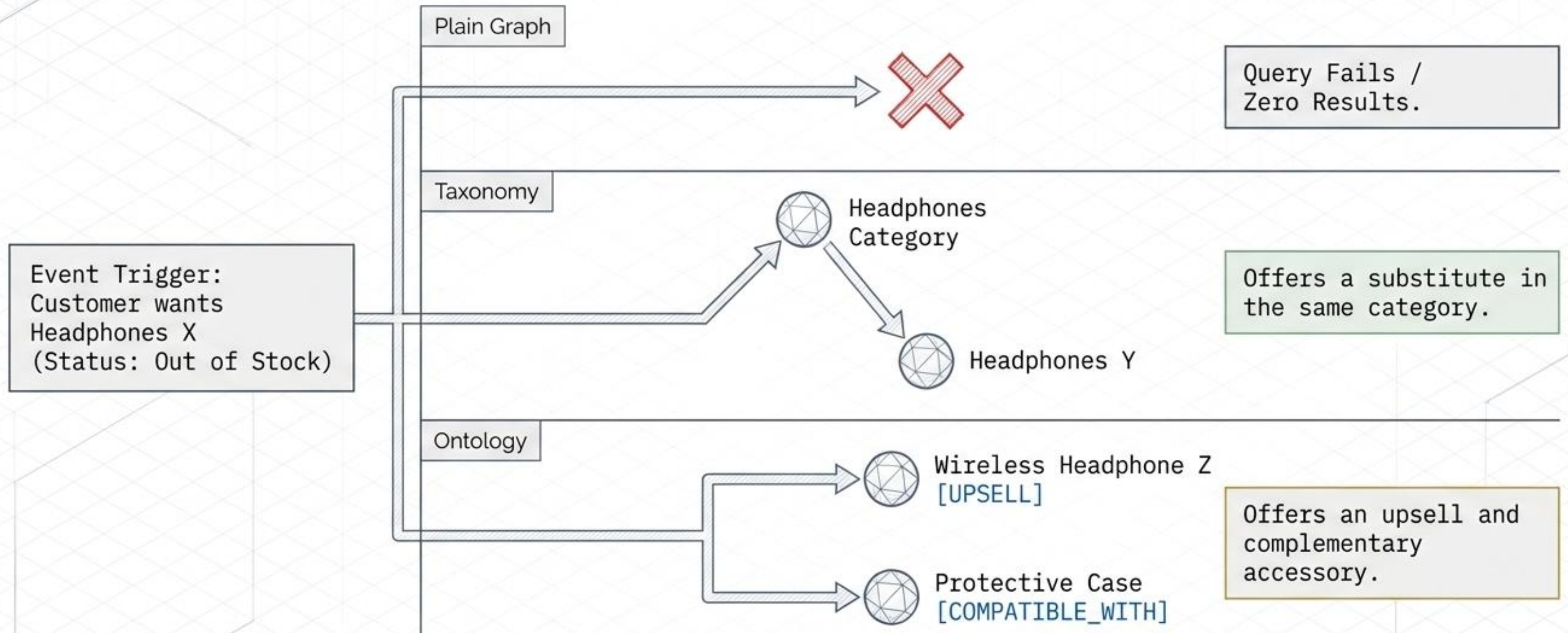
The Upgrade

Beyond vertical hierarchy. Ontologies map horizontal rules across cross-cutting concerns, enabling the derivation of new facts without the software having to understand the underlying domain.

Business Capability

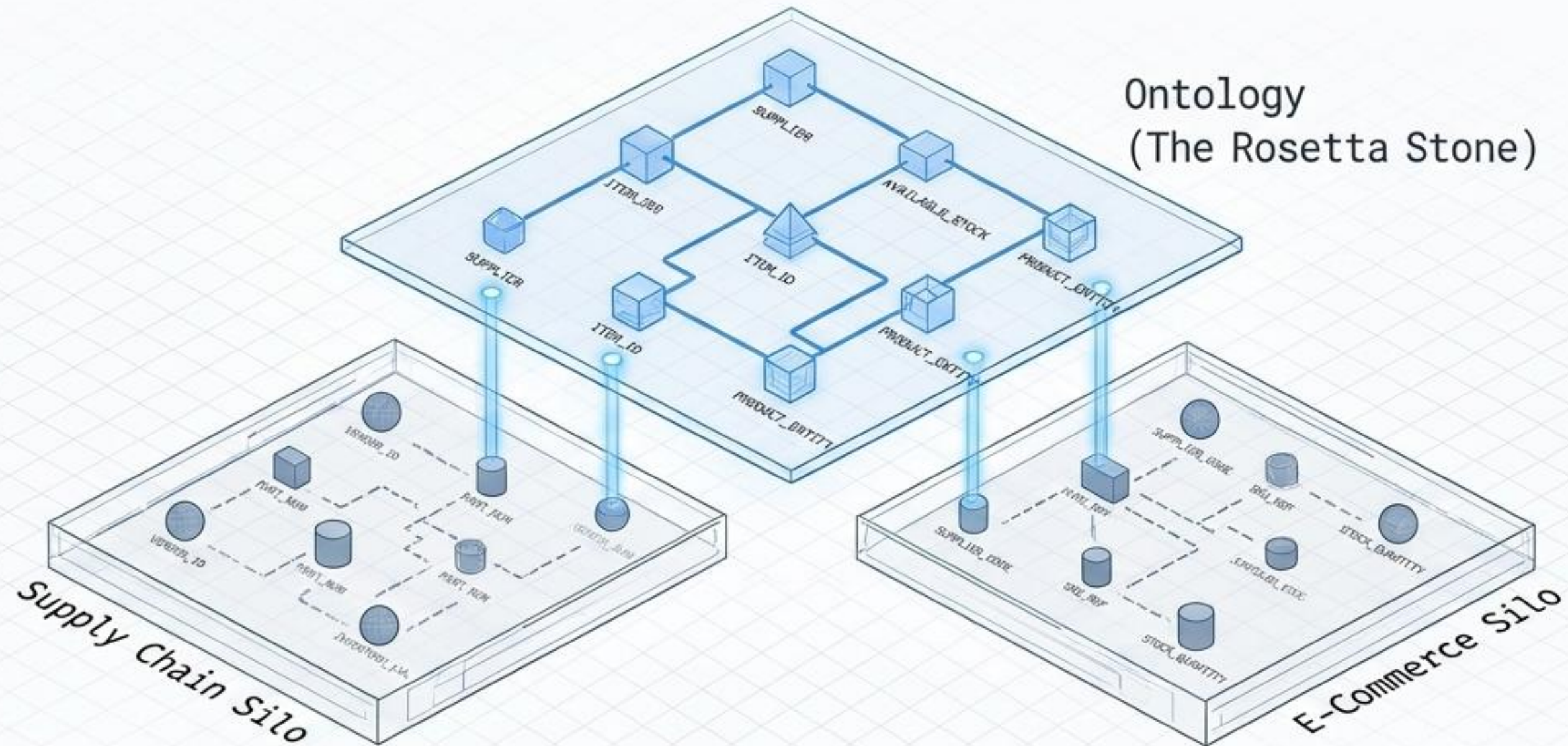
Advanced recommendation engines, upgrade paths, and cross-domain reasoning (e.g., shifting from 'what gadgets complement' to 'help me get the things I need').

The Value of Semantics in Action



Smarter data means moving **logic** from algorithms **directly into the graph**.
Every layer added transforms an operational failure into a new revenue opportunity.

The Semantic Bridge



Mechanism

By defining **cross-equivalence**, an ontology acts as a **computational contract** linking identical concepts stored under different names in separate databases.

Impact

Allows **queries** to traverse the entirety of a business domain seamlessly, mapping fragmented silos back to a single, **well-understood vocabulary**.

Organizing Principles Capability Matrix

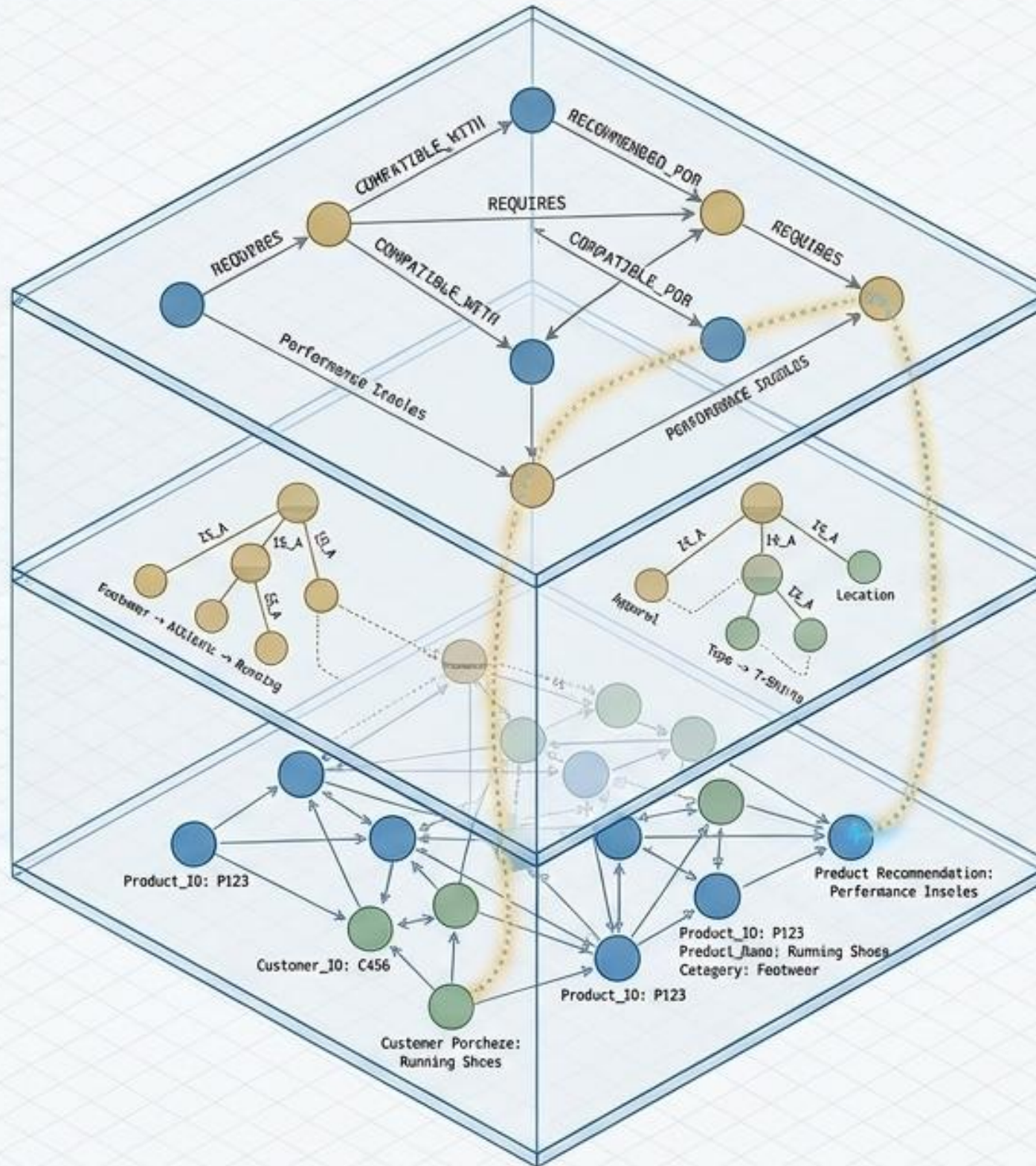
	<u>Level 0: Plain Graph</u>	<u>Level 1: Property Graph</u>	<u>Level 2: Taxonomy</u>	<u>Level 3: Ontology</u>
Organizing Principle	<u>None</u>	<u>Labels & Properties</u>	<u>Hierarchy</u>	<u>Complex Associativity</u>
Primary Relationship	Undefined	Point-to-Point	Broader/Narrower (IS_A)	Multi-dimensional (DEPENDS_ON)
Logic Location	<u>Application Code</u>	<u>Database Structure</u>	<u>Categorical Layer</u>	<u>Semantic Bridge</u>
Business Capability	Basic Queries	Visual Data Extraction	Substitution & <u>Browsing</u>	Reasoning & <u>Recommendations</u>

The Full Stack: Stacking the Layers

Custom Ontologies
(Recommendation, compatibility)

Ontologies/Taxonomies
(Product classification, promotions)

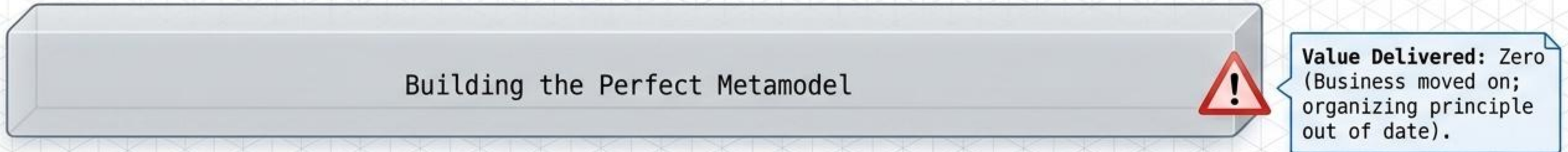
Instance Data
(Product Catalog)



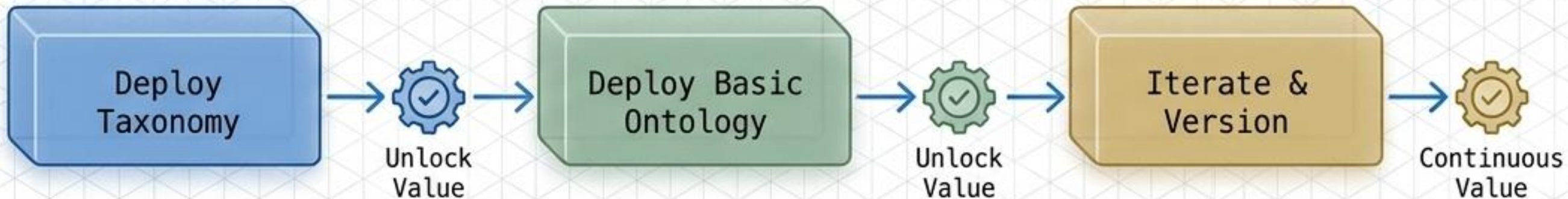
Each layer in the graph can be queried independently. When composed together, they provide the ability to execute sophisticated reasoning across multiple domains simultaneously.

Strategy: Embrace "Just-Enough Semantics"

The Trap: Boiling the Ocean



The Solution: Agile Implementation



The Monolith Trap

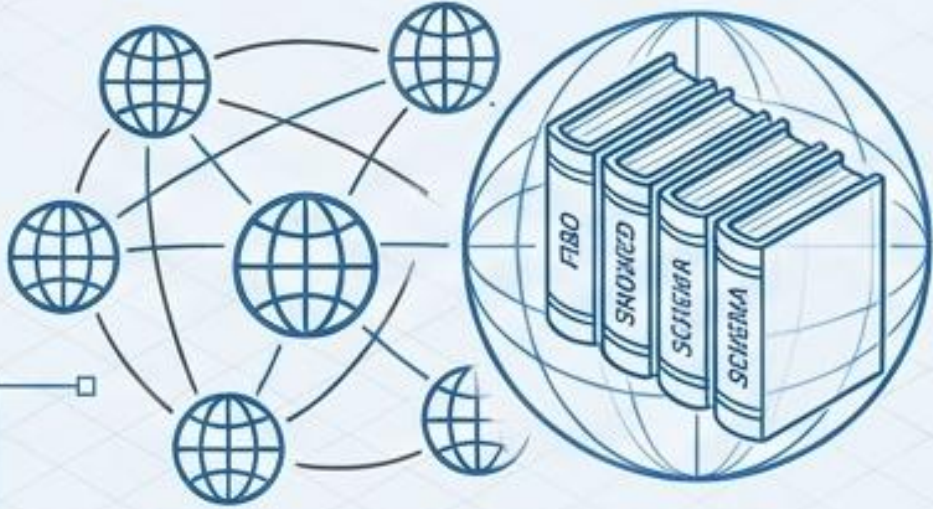
Aiming for an overly ambitious metamodel upfront is expensive and overengineered. A simple taxonomy is often enough to deliver immediate capabilities.

Iterative Value

Introduce semantic metadata only for current use cases. Treat the model as malleable and versionable. Avoid ontological perfectionism.

Standards vs. Custom Architectures

Adopt Off-the-Shelf Standards



- **Examples:** FIBO (Finance), SNOMED CT (Clinical), Schema.org (Web).
- **Why:** Fosters interoperability, knowledge reuse, and regulatory compliance.
- **Best Use:** Communicating across separate entities, such as fragmented supply chains.

Create Your Own

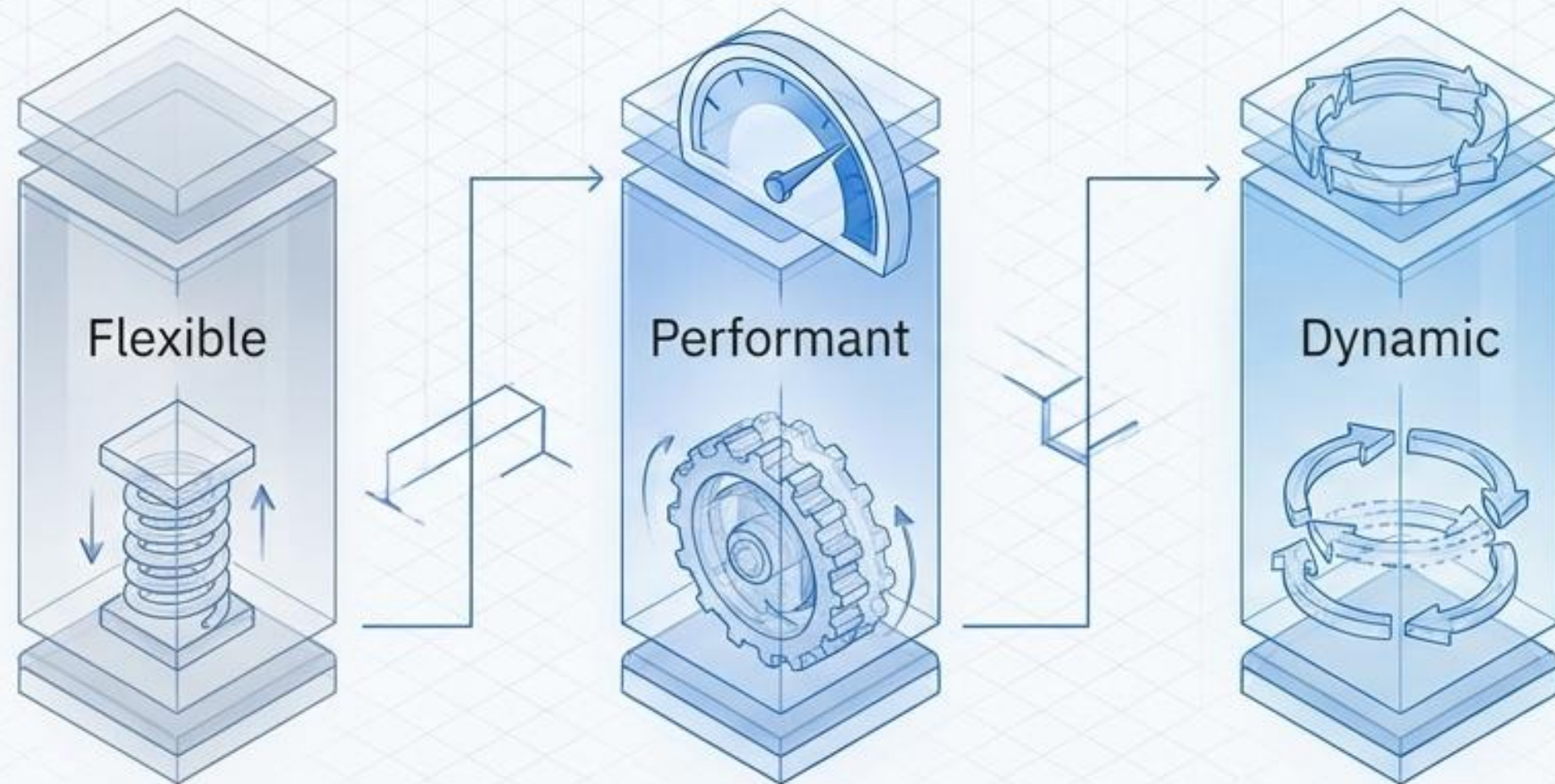


- **Tools:** OWL & RDF Schema (Ontologies), SKOS (Taxonomies).
- **Why:** Necessary when no standard fits, or fine-grained control is required.
- **Advantage:** Standard machine-readable languages provide automated software support and reasoning capabilities.

□ □ □

“The Ideal Mix: Successful organizations leverage base standards but layer on adaptive customization to keep pace with changing business realities.”

The Engine: Essential System Characteristics



ADAPTABILITY MODULE

Avoid cumbersome technologies. The graph must be easy to maintain and evolve as the business moves on.

LATENCY OPTIMIZER

Slow systems fall into disuse. A fast, up-to-date architecture empowers users and development teams.

INTERACTION CYCLE

Knowledge graphs are not static. Human and system interactions must continually enrich the data in a virtuous cycle.



Myth vs. Reality: The RDF Clarification

RDF (Resource Description Framework) is a model for data exchange (serialization), not a strict requirement for storage or querying.

Your internal data model should not be dictated by your wire format.

Labeled property graphs are fully capable and overwhelmingly preferred by the market.

The Ultimate Output: Actionable Knowledge

